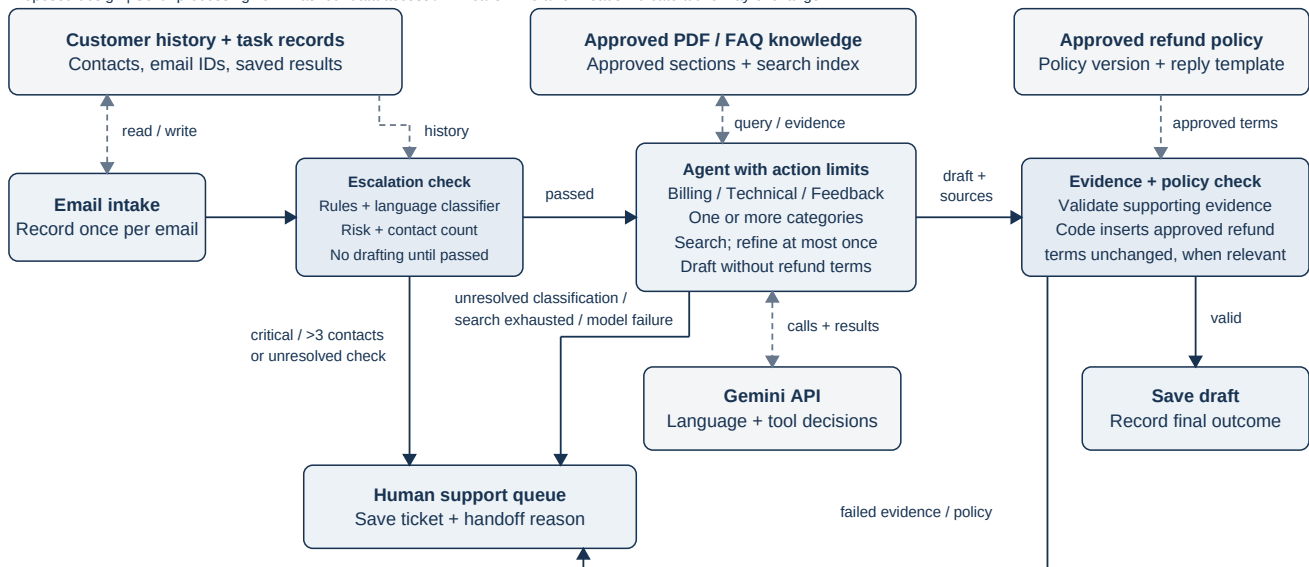


Agentic Architect Challenge

Jerence | Developer Intern assessment | Python + Gemini

Part 1 | Customer-support email architecture

Proposed design | Solid: processing flow. Dashed: data access / API calls. Two arrowheads indicate a two-way exchange.



Escalate BEFORE drafting: data loss, service outage, security breach, or >3 contacts in 7 days.

Assumptions: current email counts; unknown identity or missing history also requires a human. No automatic sending or refund approval.

Part 1: email system design

Before drafting, flag emails that mention data loss, a service outage or a security breach. Also escalate when a customer has contacted support more than three times in seven days. Drafting requires a saved decision marked passed, with completed risk and history checks. Missing history or uncertain risk sends the case to a human. The design assumes that the current email counts, both ends of the UTC time window are included and retries keep the same message ID. Save contacts and outcomes without duplicates. Escalated emails are classified after handoff, by staff if needed. The agent can refine a knowledge search once. When relevant, code inserts approved refund terms unchanged, using a recorded policy version. Missing or conflicting policy blocks the reply.

Part 2: website scraping and summaries

The assumed starting script misses JavaScript content and sends too much unrelated text in one request. The improved script extracts article content and tables, uses Playwright when little text is found and limits HTML to 2 MiB. It processes up to 100 chunks. Repeated summarisation keeps each content input within 6,000 characters. If needed, Gemini is asked once to shorten the final summary, then code enforces the 120-word limit. Long articles need more calls and processing time. Larger inputs are rejected. Login pages and infinite scrolling are not supported.

Part 3: document question-answering agent

The agent gives Gemini the full fictional ClearDesk handbook and keeps twelve completed conversation turns in memory. Gemini decides when to use the calculator. Python performs the arithmetic and displays the actual tool results. Each question allows two tool calls and three model requests, excluding SDK retries. Document answers need an existing section ID and a matching quote. Memory answers quote verified user messages. Answers that fail these checks are withheld. Reset clears memory.

Trade-offs, failures and checks

Direct Gemini API calls keep the code simple. One short document does not need a vector database. Human handoffs reduce automation, and approved refund templates need maintenance. Each Gemini attempt has a 30-second timeout. Selected temporary errors allow up to two attempts, while quota errors are not retried automatically. Failed chat turns are not saved. Part 3 provides optional diagnostic logs that exclude prompts and API keys. Offline tests check processing, summary limits and JavaScript rendering. Live tests evaluate Gemini separately. A matching quote does not prove an answer is correct. Part 1 is a proposed design, and production performance has not been measured. Measure response times, quotas and queue waiting times before deciding how many tasks can run at once.